KNOWLEDGE-BASED AGENTS

How does an agent combine what it has sensed *now* with what it *knows in general* to figure out what to do?

Knowledge-Based Agents (KBAs)

- Knowledge-Based Agents are agents that explicitly store knowledge about the world in a Knowledge Base (KB) and use an inference mechanism to derive new knowledge and decide actions.
- The **Knowledge Base (KB)** is a set of *sentences* in some formal language, known by a knowledge-based agent. The agent updates it with percepts and queries it to choose actions.
- The knowledge could be stored in logic, probability distributions, semantic networks, etc.

Examples

1. An agent navigating through a maze (A logical KBA)

KB: Propositional logic sentences describing which cells have obstacles, which cells are safe/unsafe, which cells result in reward/penalty, etc.

Inference: Uses logical entailment (model checking, resolution) to deduce safe squares.

2. Medical Diagnosis System with Bayesian Networks (A probabilistic KBA)

KB: Conditional probability tables (CPTs)

Inference: Uses Bayes' theorem to update probabilities of diseases given symptoms.

3. Plant Identification Assistant (Semantic network based KBA)

KB: Semantic network representing plant categories and their properties (typically a directed graph with concepts as vertices and relations between them as edges).

Inference: Uses graph traversal and relation chaining.

Functions used in a Knowledge-Based Agent program

- TELL: A function to add new sentences to the knowledge base.
- ASK: A function to query what is known.
- MAKE-PERCEPT-SENTENCE: A function that constructs a sentence asserting that the agent perceived the given percept at the given time.
- MAKE-ACTION-QUERY: A function that constructs a sentence asking what action should be done at the current time.
- MAKE-ACTION-SENTENCE: A function that constructs a sentence asserting that the chosen action was executed.

Knowledge-Based Agent Program

function KB-AGENT(*percept*) **returns** an *action*
  persistent: *KB*, a knowledge base
    t, a counter indicating time (initially 0)
  TELL(*KB*, MAKE-PERCEPT-SENTENCE(*percept*, *t*))
  action ← ASK(*KB*, MAKE-ACTION-QUERY(*t*))
  TELL(*KB*, MAKE-ACTION-SENTENCE(*action*, *t*))
  $t \leftarrow t + 1$
  return *action*

Logical Agents

Logical agents are knowledge-based agents **that store and reason with knowledge in formal logic**.

Why logic?

- Clarity (formal syntax & semantics)
- Supports valid reasoning

Truth tables for the five logical connectives

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
true	*true*	*false*	*true*	*true*	*true*	*true*
true	*false*	*false*	*false*	*true*	*false*	*false*
false	*true*	*true*	*false*	*true*	*true*	*false*
false	*false*	*true*	*false*	*false*	*true*	*true*

Standard logical equivalences:

Commutativity of ∧:	$P \wedge Q \equiv Q \wedge P$
Commutativity of ∨:	$P \vee Q \equiv Q \vee P$
Associativity of ∧:	$(P \wedge Q) \wedge R \equiv P \wedge (Q \wedge R)$
Associativity of ∨:	$(P \vee Q) \vee R \equiv P \vee (Q \vee R)$
Double-Negation Elimination:	$\neg(\neg P) \equiv P$
Contraposition:	$(P \Rightarrow Q) \equiv (\neg Q \Rightarrow \neg P)$
Implication Elimination:	$(P \Rightarrow Q) \equiv (\neg P \vee Q)$
Biconditional Elimination:	$(P \Leftrightarrow Q) \equiv ((P \Rightarrow Q) \wedge (Q \Leftarrow P))$
De Morgan Law:	$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$
De Morgan Law:	$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$
Distributivity of ∧ over ∨:	$P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$
Distributivity of ∨ over ∧:	$P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$

And Elimination:

$$P \wedge Q \Rightarrow P \text{ and } P \wedge Q \Rightarrow Q$$

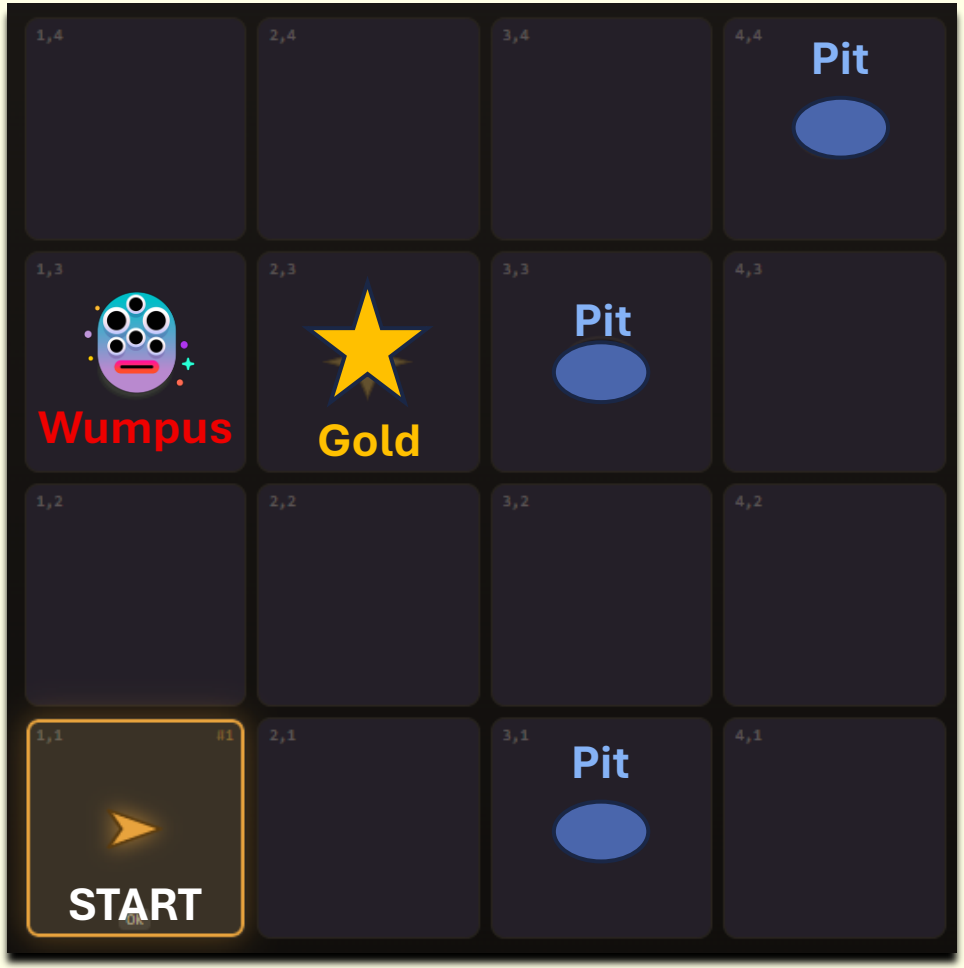
Modus Ponens:

$$(P \Rightarrow Q) \wedge P \Rightarrow Q$$

Unit Resolution Rule:

$$(P \vee Q) \wedge \neg P \Rightarrow Q$$

The Wumpus World



- The Wumpus World is a cave consisting of rooms connected by passageways.
- One of the rooms contains the “Wumpus”, a beast that eats anyone who enters its room.
- The agent can shoot the Wumpus with an arrow; and the agent has only one arrow.
- Some rooms contain bottomless pits that will trap anyone who enters these rooms (except for the Wumpus).
- There is a heap of gold in one of the rooms, which the agent aims to collect and climb out of the cave.

PEAS Description of the Wumpus World

Performance Measure:

+1000 for climbing out of the cave with the gold.

–1000 for falling into a pit or being eaten by the Wumpus.

–1 for each action taken.

–10 for using up the arrow.

The game ends either when the agent dies or when the agent climbs out of the cave.

Environment:

A 4×4 grid of rooms, with walls surrounding the grid.

The agent starts in the square labeled [1,1], facing to the east.

There is one cell with the gold and one cell with the Wumpus. The locations of the gold and the Wumpus are chosen randomly, with a uniform distribution, from the squares other than the start square.

Each square other than the start can be a pit, with probability 0.2.

Actuators:

To move, the agent can take the actions ***Forward***, ***Turn Left***, ***Turn Right***.

The agent dies if it enters a square containing a pit or a live Wumpus.

If an agent tries to move forward and bumps into a wall, then the agent does not move.

The action ***Grab*** can be used to pick up the gold if it is in the same square as the agent.

The action ***Shoot arrow*** can be used to fire an arrow in a straight line in the direction the agent is facing. The arrow continues until it either hits (and hence kills) the Wumpus or hits a wall.

The agent has only one arrow, so only the first *Shoot* action has any effect.

The action ***Climb out*** can be used to climb out of the cave, but only from square [1,1].

Sensors:

The agent perceives a ***Stench*** in the squares directly (not diagonally) adjacent to the Wumpus.

The agent perceives a ***Breeze*** in the squares directly adjacent to a pit.

The agent perceives a ***Glitter*** in the square where the gold is.

The agent perceives a ***Bump*** when an agent walks into a wall.

When the Wumpus is killed, it emits a ***Scream*** that can be perceived anywhere in the cave.

The percepts are given to the agent program as a list of five symbols (bits) corresponding to *Stench, Breeze, Glitter, Bump, Scream*.

For example, if there is a *Stench* and a *Breeze*, but no *Glitter*, *Bump* or *Scream*, then the percept is:

[*Stench, None, None, Bump, None*].

The Knowledge Base (KB):

We will use the following notations:

For every $[x, y]$ location,

- if there is a pit in $[x, y]$, $P_{x,y}$ is true.
- if the Wumpus is in $[x, y]$ (alive or dead), $W_{x,y}$ is true.
- if there is a *Breeze* in $[x, y]$, $B_{x,y}$ is true.
- if there is a *Stench* in $[x, y]$, $S_{x,y}$ is true.
- if the agent is in $[x, y]$, $L_{x,y}$ is true.

The Knowledge Base (KB):

- There is no pit in [1,1].

$$R_1: \neg P_{11}$$

- A square is breezy if and only if there is a pit in a neighbouring square.

$$R_2: B_{11} \Leftrightarrow P_{12} \vee P_{21}$$
$$R_3: B_{12} \Leftrightarrow P_{11} \vee P_{22} \vee P_{13}$$

  and so on.

- A square has stench if and only if there is Wumpus in a neighbouring square.

$$R_4: S_{11} \Leftrightarrow W_{12} \vee W_{21}$$
$$R_5: S_{12} \Leftrightarrow W_{11} \vee W_{22} \vee W_{13}$$

  and so on.

Inferring $\neg P_{21}$ (There is no pit in [2,1]):

Applying biconditional elimination on $B_{11} \Leftrightarrow P_{12} \vee P_{21}$ gives

$$R_6\colon (B_{11} \Rightarrow P_{12} \vee P_{21}) \wedge (P_{12} \vee P_{21} \Rightarrow B_{11})$$

Applying and-elimination to R_6 gives

$$R_7\colon P_{12} \vee P_{21} \Rightarrow B_{11}$$

Using logical equivalence for contrapositives,

$$R_8\colon \neg B_{11} \Rightarrow \neg(P_{12} \vee P_{21})$$

Applying modus ponens on R_8 and the percept $\neg B_{11}$,

$$R_9\colon \neg(P_{12} \vee P_{21})$$

Applying De Morgan's law on R_9,

$$R_{10}\colon \neg P_{12} \wedge \neg P_{21}$$

Applying and-elimination on R_{10},

$$R_{11}\colon \neg P_{21}$$

Time step t	KB	Percept(s)	Inferences	Action
0	Rules of the game	$\neg B_{11}, \neg S_{11}$	$\neg P_{12}, \neg P_{21}, \neg W_{12}, \neg W_{21}$	*Forward* ([2,1])
1	Rules of the game, $\neg B_{11}, \neg S_{11}, \neg P_{12}, \neg P_{21}, \neg W_{12}, \neg W_{21}$	$B_{21}, \neg S_{21}$	$P_{31} \vee P_{22}, \neg W_{31}, \neg W_{22}$	*Turn Left, Turn Left, Forward* ([1,1]) *Turn Right, Forward* ([1,2])
2	Rules of the game, $\neg B_{11}, \neg S_{11}, \neg P_{12}, \neg P_{21}, \neg W_{12}, \neg W_{21}, B_{21}, \neg S_{21}, P_{31} \vee P_{22}, \neg W_{31}, \neg W_{22}$	$\neg B_{12}, S_{12}$	$W_{13}, \neg P_{22}, \neg P_{13}, P_{31}$	*Turn Right, Forward* ([2,2])

Time step t	KB	Percept(s)	Inferences	Action
3	Rules of the game, $\neg B_{11}, \neg S_{11}, \neg P_{12}, \neg P_{21}, \neg W_{12}, \neg W_{21}, B_{21}, \neg S_{21}, P_{31} \vee P_{22}, \neg W_{31}, \neg W_{22} \neg B_{12}, S_{12}, W_{13}, \neg P_{22}, \neg P_{13}, P_{31}$	$\neg B_{22}, \neg S_{22}$	$\neg P_{23}, \neg W_{23}, \neg P_{32}, \neg W_{32}$	*Turn Left*, *Forward* ([2,3])
4	Rules of the game, $\neg B_{11}, \neg S_{11}, \neg P_{12}, \neg P_{21}, \neg W_{12}, \neg W_{21}, B_{21}, \neg S_{21}, P_{31} \vee P_{22}, \neg W_{31}, \neg W_{22} \neg B_{12}, S_{12}, W_{13}, \neg P_{22}, \neg P_{13}, P_{31}, \neg B_{22}, \neg S_{22}, \neg P_{23}, \neg W_{23}, \neg P_{32}, \neg W_{32}$	B_{23}, S_{23}, G_{23}	$P_{33} \vee P_{24} \vee P_{13}$	*Grab* *Turn Left*, *Turn Left*, *Forward* ([2,2]) *Turn Right*, *Forward* ([1,2]) *Turn Left*, *Forward* ([1,1]) *Climb out*

Model

An assignment of a truth value to every propositional symbol (a “possible world”).

Example:

P: Today is Monday.

Q: There is a quiz scheduled for today.

{P = *true*, Q = *false*} is a model.

Note: When there are n propositional symbols, the number of possible models is 2^n.

Entailment

A sentence P "entails" a sentence Q (written as $P \vDash Q$) if in every model in which sentence P is true, sentence Q is also true.

The statement $P \vDash Q$ is logically equivalent to the statement that $P \Rightarrow Q$ is a tautology (which means $P \Rightarrow Q$ is **always** true, that is, true **in all possible models**).

Example 1:

P: It is raining.
Q: It is sunny.

$P \vee Q$: It is raining OR it is sunny.

$P \vDash P \vee Q$
In every model where P is true, $P \vee Q$ is also true.

Checking whether $P \models P \vee Q$:

KB: P
Query: $P \vee Q$

P	Q	$P \vee Q$	$P \Rightarrow P \vee Q$
T	T	T	T
T	F	T	T
F	T	T	T
F	F	F	T

In every model where P is true, $P \vee Q$ is also true.
Hence $P \models P \vee Q$.

Example 2:

KB:
$(P \wedge \neg Q) \Rightarrow R$
P
$\neg Q$

α: R

$KB \models \alpha$
In every model where KB is true, α is also true.

Checking whether KB ⊨ R :

KB: $((P \wedge \neg Q) \Rightarrow R) \wedge P \wedge (\neg Q)$
Query: R

P	Q	R	¬Q	(P ∧ ¬Q) ⇒ R	KB	KB ⇒ R
T	T	T	F	T	F	T
T	T	F	F	T	F	T
T	F	T	T	T	T	T
T	F	F	T	F	F	T
F	T	T	F	T	F	T
F	T	F	F	T	F	T
F	F	T	T	T	F	T
F	F	F	T	T	F	T

In every model where KB is true, R is also true.
Hence **KB ⊨ R**.

Example 3:

KB:
All men are mortal.
Socrates is a man.

α: Socrates is mortal.

$$\{\forall x\,(Man(x) \Rightarrow Mortal(x)),\ Man(Socrates)\} \vDash Mortal(Socrates)$$

$$\text{KB} \vDash \alpha$$

In every model where KB is true, α is also true.

Non-examples:

P: 2 is odd
Q: Paris is in France
P does not entail Q, although P ⇒ Q is (vacuously) true in the 'usual' world.

P: n is divisible by 4.
Q: n is even.
P does not entail Q, although P ⇒ Q is true when the usual axioms and definitions of arithmetic are assumed (that is, in the 'world of usual arithmetic').

Entailment does not care what the words/predicates in the sentences “actually mean”.

It only cares about the structural pattern.

KB $\models \alpha$ means the following:
For **every** possible model *(even any bizarre reinterpretation you could think of)*, if that model satisfies KB, it is forced to also satisfy α.

In other words, KB $\models \alpha$ if and only if there is **no model** that satisfies KB but fails to satisfy α.

Entailment only exists when there is a *necessary logical link*.

Otherwise, changing interpretations (models) will break it.

Let $M(\alpha)$ and $M(\beta)$ be the sets of models in which α and β are true respectively.

Then $\alpha \vDash \beta$ if and only if $M(\alpha) \subseteq M(\beta)$.

Model checking

A way of checking whether $\alpha \vDash \beta$ by checking whether $M(\alpha) \subseteq M(\beta)$.

Suppose we want to check whether $KB \vDash \alpha$, where KB and α contain n propositional symbols altogether.

Note that an algorithm that generates and stores all 2^n possible models to check each of them takes $O(n.2^n)$ space.

The following TT-ENTAILS algorithm is a recursive algorithm that takes only $O(n)$ space by generating models one at a time through a depth-first-search over the binary tree of models.

A general algorithm to decide entailment in propositional logic

function TT-ENTAILS?(*KB*, α) **returns** *true* or *false*
  inputs: *KB*, the knowledge base, a sentence in propositional logic
    α, the query, a sentence in propositional logic
  symbols ← a list of the proposition symbols in *KB* and α
  return TT-CHECK-ALL(*KB*, α, *symbols*, {})

function TT-CHECK-ALL(*KB*, α, *symbols*, *model*) **returns** *true* or *false*
  if EMPTY?(*symbols*) **then**
    if PL-TRUE?(*KB*, *model*) **then return** PL-TRUE?(α, *model*)
    else return *true* (When KB is false, always return true)
  else
    P ← FIRST(*symbols*)
    rest ← REST(*symbols*)
    return (TT-CHECK-ALL(*KB*, α, *rest*, *model* ∪ {*P* = *true*})
      and
      TT-CHECK-ALL(*KB*, α, *rest*, *model* ∪ {*P* = *false*}))

Example 1:

$KB = A \wedge B,\ \alpha = A$

TT-ENTAILS?(KB, α)
symbols $\leftarrow [A, B]$

TT-CHECK-ALL$(KB, \alpha, [A, B], \{\})$

symbols not empty $\rightarrow P = A$, *rest* $= [B]$

- TT-CHECK-ALL$(KB, \alpha, [B], \{A = true\})$ $\rightarrow$ **Branch 1**
- TT-CHECK-ALL$(KB, \alpha, [\mathrm{B}], \{A = false\})$ $\rightarrow$ **Branch 2**

Result = (**Branch 1**) AND (**Branch 2**)

(Continued on next page)

Branch 1:

TT-CHECK-ALL(KB, α, [B], $\{A = true\}$)

symbols not empty → $P = B$, *rest* = []

- TT-CHECK-ALL(KB, α, [], $\{A = true, B = true\}$)
  - *symbols* empty
  - PL-TRUE?(KB) = $T \wedge T = true$
  - → return PL-TRUE?(α) = A = ***true***
- TT-CHECK-ALL(KB, α, [], $\{A = true, B = false\}$)
  - *symbols* empty
  - PL-TRUE?(KB) = $T \wedge F = false$
  - → KB is *false*, return ***true***

true AND ***true*** = ***true***

(Continued on next page)

Branch 2:

TT-CHECK-ALL(KB, α, [B], $\{A = false\}$)

symbols not empty $\rightarrow$ $P = B$, *rest* = []

TT-CHECK-ALL(KB, α, [], $\{A = false, B = true\}$)

symbols empty

PL-TRUE?(KB) = $F \wedge T = false$

$\rightarrow$ KB is *false*, return ***true***

TT-CHECK-ALL(KB, α, [], $\{A = false, B = false\}$)

symbols empty

PL-TRUE?(KB) = $F \wedge F = false$

$\rightarrow$ KB is *false*, return ***true***

true AND ***true*** = ***true***

(Continued on next page)

Back at the root (initial call):

true (**Branch 1**) AND ***true*** (**Branch 2**) = ***true***

So TT-ENTAILS?($A \wedge B,\ A$) returns ***true***, that is, $A \wedge B \models A$.

Example 2:

$KB = A \vee B,\ \alpha = A$

TT-ENTAILS?(KB, α)
symbols $\leftarrow [A, B]$

TT-CHECK-ALL$(KB, \alpha, [A, B], \{\})$

symbols not empty $\rightarrow P = A$, *rest* $= [B]$

- TT-CHECK-ALL$(KB, \alpha, [B], \{A = true\})$ $\rightarrow$ **Branch 1**
- TT-CHECK-ALL$(KB, \alpha, [\mathrm{B}], \{A = false\})$ $\rightarrow$ **Branch 2**

Result = (**Branch 1**) AND (**Branch 2**)

(Continued on next page)

Branch 1:

TT-CHECK-ALL(KB, α, [B], $\{A = true\}$)

symbols not empty → $P = B$, *rest* = []

TT-CHECK-ALL(KB, α, [], $\{A = true, B = true\}$)

symbols empty

PL-TRUE?(KB) = $T \vee T = true$

→ return PL-TRUE?(α) = A = ***true***

TT-CHECK-ALL(KB, α, [], $\{A = true, B = false\}$)

symbols empty

PL-TRUE?(KB) = $T \vee F = true$

→ return PL-TRUE?(α) = A = ***true***

true AND ***true*** = ***true***

(Continued on next page)

Branch 2:

TT-CHECK-ALL(KB, α, [B], $\{A = false\}$)

symbols not empty $\rightarrow$ $P = B$, *rest* = []

TT-CHECK-ALL(KB, α, [], $\{A = false, B = true\}$)

symbols empty

PL-TRUE?(KB) = $F \vee T = true$

$\rightarrow$ return PL-TRUE?(α) = $A =$ ***false***

TT-CHECK-ALL(KB, α, [], $\{A = false, B = false\}$)

symbols empty

PL-TRUE?(KB) = $F \vee F = false$

$\rightarrow$ KB is *false*, return ***true***

false AND ***true*** = ***false***

(Continued on next page)

Back at the root (initial call):

true (**Branch 1**) AND ***false*** (**Branch 2**) = ***false***

So TT-ENTAILS?$(A \vee B, A)$ returns ***false***, that is, $A \vee B \not\models A$.

Hence, TT-ENTAILS?(KB, α) returns the truth value of the following statement:
For every model m, $m \in M(KB) \Rightarrow m \in M(\alpha)$;
that is, $M(KB) \subseteq M(\alpha)$,
which is equivalent to KB $\vDash \alpha$.

The time complexity of TT-ENTAILS? is $O(2^n)$, where n is the total number of propositional symbols in KB and α.

Note that TT-ENTAILS? cannot detect failures early.

For example, in the $A \wedge B \vDash A$ trace, once $A = false$ is assigned, $KB = A \wedge B$ is already *false* whatever B is. The algorithm still branches on B and evaluates two leaves, only to find "*KB false*, return *true*" both times.

In general, the algorithm always builds out every branch and checks all 2^n complete models before answering.

This is because TT-ENTAILS evaluates *KB* only after every symbol has a value, and never looks at *KB* with partial models.

We need a cheap test on a partial model: “Is *KB* already *false*, or already *true*?”

Conjunctive Normal Form (CNF) makes this test trivial.

A CNF is built from ***literals*** and ***clauses***.

Literals

A literal is either an atomic sentence (a positive literal) or a negated atomic sentence (a negative literal).

Examples:

P, Q, W_{31} are positive literals.
$\neg P$, $\neg Q$, $\neg P_{12}$ are negative literals.

Clause

A disjunction of literals is called a clause.

Examples:

$P \vee Q \vee R$
$P \vee Q \vee \neg R \vee S$
$P_{13} \vee P_{22}$

The Unit Resolution Rule

$$(l_1 \vee l_2 \vee \cdots \vee l_k) \wedge m \models l_1 \vee l_2 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \cdots \vee l_k$$

where each l_j $(1 \leq j \leq k)$ is a literal
(hence $l_1 \vee l_2 \vee \cdots \vee l_k$ and $l_1 \vee l_2 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \cdots \vee l_k$ are clauses),
m is a literal,
l_i and m are complementary literals (negations of each other).

Modus ponens is also a case of Unit Resolution:

$(P \rightarrow Q) \wedge P \models Q$ (Modus ponens) is the same as:

$(\neg P \vee Q) \wedge P \models Q$

The Full Resolution Rule

$$(l_1 \vee l_2 \vee \cdots \vee l_k) \wedge (m_1 \vee m_2 \vee \cdots \vee m_n)$$
$$\vDash\ l_1 \vee l_2 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \cdots \vee l_k \vee m_1 \vee m_2 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n$$

where each l_r $(1 \leq r \leq k)$, m_s $(1 \leq s \leq n)$ is a literal,
l_i and m_j are complementary literals (negations of each other).

Example:
$k = 2$ and $n = 2$:
$(P \vee Q) \wedge (\neg P \vee R) \vDash Q \vee R$

Proof:

Suppose $(P \vee Q) \wedge (\neg P \vee R)$ is *true*.
Then both $(P \vee Q)$ and $(\neg P \vee R)$ are *true*.
To show: $Q \vee R$ is *true*.

Case-1: Suppose P is *false*. Then since $P \vee Q$ is *true*, Q must be *true*.
Case-2: Suppose P is *true*. Then since $\neg P \vee R$ is *true*, R must be *true*.

Hence one of Q and R is *true*. Therefore $Q \vee R$ is *true*.

Verification by model checking using truth table:

P	Q	R	$P \vee Q$	$\neg P \vee R$	$(P \vee Q) \wedge (\neg P \vee R)$	$Q \vee R$	$(P \vee Q) \wedge (\neg P \vee R) \Rightarrow (Q \vee R)$
T	T	T	T	T	T	T	T
T	T	F	T	F	F	T	T
T	F	T	T	T	T	T	T
T	F	F	T	F	F	F	T
F	T	T	T	T	T	T	T
F	T	F	T	T	T	T	T
F	F	T	F	T	F	T	T
F	F	F	F	T	F	F	T

Can we resolve more than one pair of complementary literals at a time?

For example, does $(P \vee Q \vee R) \wedge (\neg P \vee \neg Q \vee S) \vDash R \vee S$?

NO.

In the model $\{P = true, Q = false, R = false, S = false\}$,
$(P \vee Q \vee R) \wedge (\neg P \vee \neg Q \vee S)$ is *true* but $R \vee S$ is *false*.

The **Resolution-based Inference Algorithm** is based on the following fact:

Every sentence in propositional logic is logically equivalent to a conjunction of clauses.

Conjunctive Normal Form (CNF):

A sentence expressed as a conjunction of clauses is said to be in Conjunctive Normal Form (CNF).

Examples:

$(P \vee Q \vee R) \wedge (R \vee \neg S)$

$(\neg P \vee Q) \wedge (P \vee R) \wedge (\neg R \vee S)$

$P \wedge (Q \vee \neg R)$

Example 1:

Conversion of $B_{11} \Leftrightarrow P_{12} \vee P_{21}$ into its Conjunctive Normal Form (CNF):

Applying Biconditional Elimination,

$$(B_{11} \Rightarrow P_{12} \vee P_{21}) \wedge (P_{12} \vee P_{21} \Rightarrow B_{11})$$

Converting conditionals to disjunctions,

$$(\neg B_{11} \vee (P_{12} \vee P_{21})) \wedge (\neg(P_{12} \vee P_{21}) \vee B_{11})$$

Using De Morgan's law,

$$(\neg B_{11} \vee P_{12} \vee P_{21}) \wedge ((\neg P_{12} \wedge \neg P_{21}) \vee B_{11})$$

Using Right Distributive law,

$$(\neg B_{11} \vee P_{12} \vee P_{21}) \wedge (\neg P_{12} \vee B_{11}) \wedge (\neg P_{21} \vee B_{11})$$

Example 2:

Conversion of $\neg(P \Rightarrow Q) \vee (Q \wedge R \wedge \neg S)$ into its CNF:

Converting conditional to disjunction,

$$\neg(\neg P \vee Q) \vee (Q \wedge R \wedge \neg S)$$

Using De Morgan's law,

$$(P \wedge \neg Q) \vee (Q \wedge R \wedge \neg S)$$

Using Left Distributive law,

$$((P \wedge \neg Q) \vee Q) \wedge ((P \wedge \neg Q) \vee R) \wedge ((P \wedge \neg Q) \vee \neg S)$$

Using Right Distributive law,

$$(P \vee Q) \wedge (\neg Q \vee Q) \wedge (P \vee R) \wedge (\neg Q \vee R) \wedge (P \vee \neg S) \wedge (\neg Q \vee \neg S)$$

As $\neg Q \vee Q \equiv \mathbf{T}$, we get

$$(P \vee Q) \wedge (P \vee R) \wedge (\neg Q \vee R) \wedge (P \vee \neg S) \wedge (\neg Q \vee \neg S)$$

Validity and Satisfiability

A sentence is said to be **valid** if it is true in all models.

- Valid sentences are called tautologies.

A sentence is said to be **satisfiable** if it is true in at least one model.

Note:

- P is valid if and only if $\neg P$ is not satisfiable.
- P is satisfiable if and only if $\neg P$ is not valid.

$\alpha \vDash \beta$

$\Leftrightarrow$ $\alpha \Rightarrow \beta$ is valid.

$\Leftrightarrow$ $\neg(\alpha \Rightarrow \beta)$ is not satisfiable.

$\Leftrightarrow$ $\neg(\neg\alpha \vee \beta)$ is not satisfiable.

$\Leftrightarrow$ $\alpha \wedge \neg\beta$ is not satisfiable.

(Proof by Contradiction)

Resolution-based Inference Algorithm

function PL-RESOLUTION(KB, α) **returns** *true* or *false*
  inputs: KB, the knowledge base, a sentence in propositional logic
    α, the query, a sentence in propositional logic

  clauses ← the set of clauses in the CNF representation of $KB \wedge \neg\alpha$
  new ← {}
  while *true* **do**
    for each pair of clauses C_i, C_j **in** *clauses* **do**
      resolvents ← PL-RESOLVE(C_i, C_j)
      if *resolvents* contains the empty clause **then return** *true*
      new ← *new* ∪ *resolvents*
    if *new* ⊆ *clauses* **then return** *false*
    clauses ← *clauses* ∪ *new*

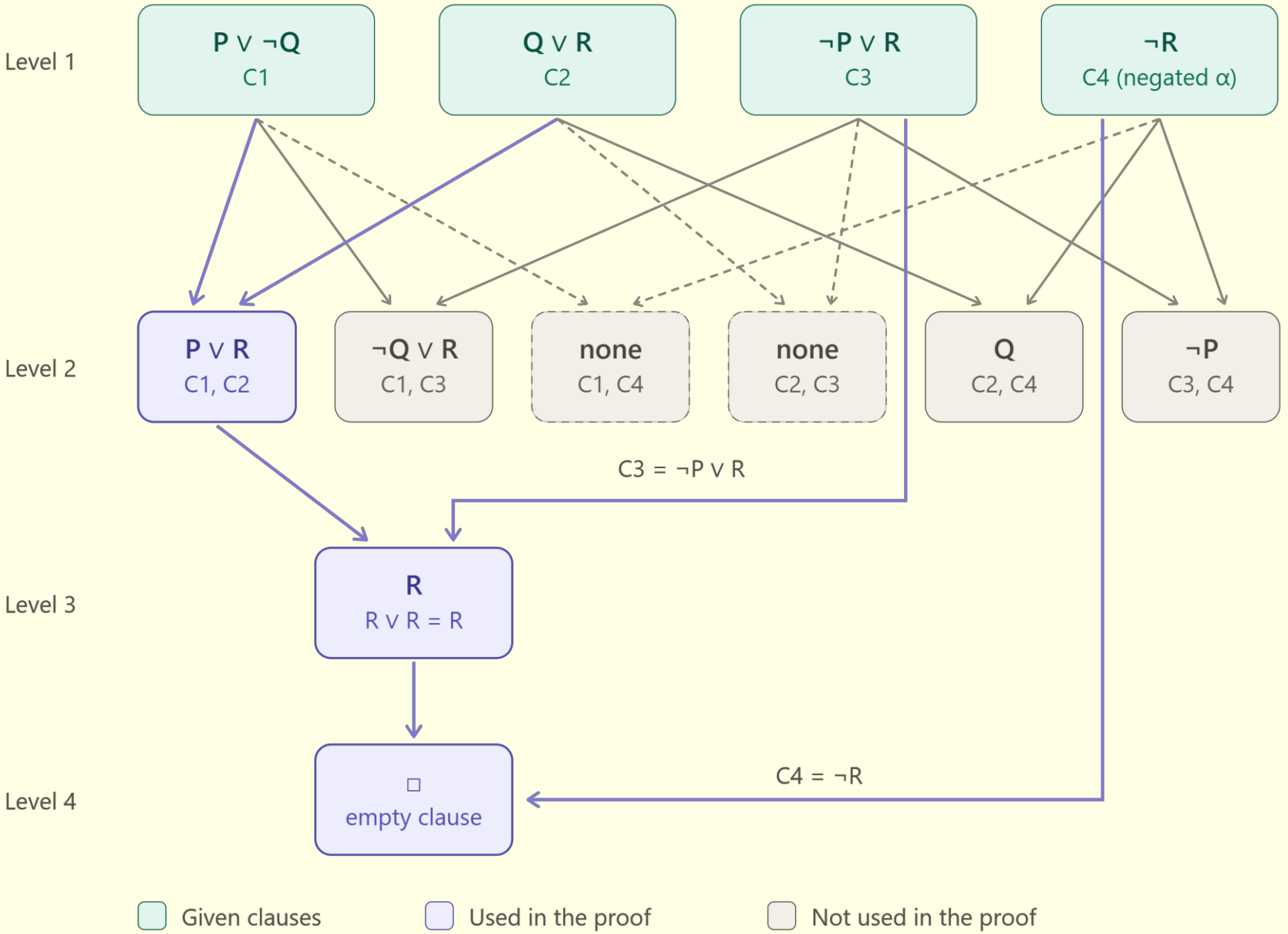
Example:

KB: $(P \vee \neg Q) \wedge (Q \vee R) \wedge (\neg P \vee R)$
α: R

clauses = $\{P \vee \neg Q, Q \vee R, \neg P \vee R, \neg R\}$

The next page shows a partial application of the Resolution-based Inference Algorithm to this example.

Level 1 is the input. Each higher level is one more round of resolution, and a resolvent may use clauses from any earlier level.


Level 1
P ∨ ¬Q
C1
Q ∨ R
C2
¬P ∨ R
C3
¬R
C4 (negated α)
Level 2
P ∨ R
C1, C2
¬Q ∨ R
C1, C3
none
C1, C4
none
C2, C3
Q
C2, C4
¬P
C3, C4
C3 = ¬P ∨ R
Level 3
R
R ∨ R = R
Level 4
□
empty clause
C4 = ¬R
Given clauses
Used in the proof
Not used in the proof

Therefore, PL-RESOLUTION returns *true*, that is, $KB \models R$.

Boolean Satisfiability (SAT) Problem

- The problem of determining if there exists an interpretation (a model) that satisfies a given Boolean formula (propositional sentence).
- The first problem to be proved to be an NP-complete problem (1970's).

Cook-Levin Theorem: The Boolean Satisfiability Problem is NP-complete.

DPLL Algorithm (named after **D**avis, **P**utnam, **L**ogemann and **L**oveland)
- To check satisfiability of a sentence in propositional logic

- Involves three improvements over the simple scheme of TT-ENTAILS? :

1. Early termination
2. Pure symbol heuristic
3. Unit clause heuristic

Early termination

The algorithm detects whether the sentence must be true or false, even with a partially completed model.

- A clause is true if any literal is true.
- A sentence is false if any clause is false.
- A sentence is true if all clauses are true.

Pure symbol heuristic

A pure symbol is a symbol that appears with the same “sign” in all the clauses.

In order for a sentence to be true, a model of the sentence must have the pure symbols assigned so as to make their literals true.

The following is a more precise definition of a pure symbol, as used by the DPLL algorithm:

Let S be a set of clauses (CNF) and let m be a partial model. A symbol P is **pure with respect to *(S, m)*** if:
1. P is unassigned in m;
2. P occurs in at least one clause of S that is not yet true in m;
3. in all such not-yet-true clauses, P occurs with the same sign (always positive or always negative).

Occurrences of P in clauses already made true by m are ignored.

Unit clause heuristic

A unit clause is a clause in which all literals except one are already assigned false by the model.

The following is a more precise definition of a unit clause, as used by the DPLL algorithm:

Let S be a set of clauses and m a partial model. A clause $C \in S$ is a **unit clause with respect to *(S, m)*** if:
1. exactly one literal l of C has its symbol unassigned in m, and
2. every other literal of C is false in m.

Then l is the **unit literal** of C.

If $l = P$, FIND-UNIT-CLAUSE returns (P, *true*).
If $l = \neg P$, FIND-UNIT-CLAUSE returns (P, *false*).
If several unit clauses exist, any one can be chosen.

DPLL Algorithm

function DPLL-SATISFIABLE?(*s*) **returns** *true* or *false*
  inputs: *s*, a sentence in propositional logic

  clauses ← the set of clauses in the CNF representation of *s*
  symbols ← a list of the proposition symbols in *s*
  return DPLL(*clauses*, *symbols*, {})

function DPLL(*clauses*, *symbols*, *model*) **returns** *true* or *false*
  if every clause in *clauses* is *true* in *model* **then return** *true*
  if some clause in *clauses* is *false* in *model* **then return** *false*
  P, *value* ← FIND-PURE-SYMBOL(*symbols*, *clauses*, *model*)
  if *P* is non-null **then return** DPLL(*clauses*, *symbols* – *P*, *model* ∪ {*P*=*value*})
  P, *value* ← FIND-UNIT-CLAUSE(*clauses*, *model*)
  if *P* is non-null **then return** DPLL(*clauses*, *symbols* – *P*, *model* ∪ {*P*=*value*})
  P ← FIRST(*symbols*)
  rest ← REST(*symbols*)
  return DPLL(*clauses*, *rest*, *model* ∪ {*P*=*true*}) **or**
      DPLL(*clauses*, *rest*, *model* ∪ {*P*=*false*}))

Example 1:
To check the satisfiability of $(A \vee B) \wedge (\neg B \vee \neg C) \wedge (C \vee A) \wedge (\neg D \vee E) \wedge (D \vee \neg E) \wedge D$:

Clauses:

C1: $A \vee B$
C2: $\neg B \vee \neg C$
C3: $C \vee A$
C4: $\neg D \vee E$
C5: $D \vee \neg E$
C6: D

Initial call: DPLL(*clauses*, $[A, B, C, D, E]$, $\{\}$)

Function Call	***symbols***	***model***	**Clause status**	**Pure symbol**	**Unit clause**	**Action (Assignment)**
1	A, B, C, D, E	{}	None *true*, None *false*	A (positive)		$A = true$
2	B, C, D, E	$\{A = true\}$	C1, C3 *true*	B (negative)		$B = false$
3	C, D, E	$\{A = true, B = false\}$	C1, C2, C3 *true*	-	D (C6)	$D = true$
4	C, E	$\{A = true, B = false, D = true\}$	C1, C2, C3, C5, C6 *true*	E (positive)		$E = true$
5	C	$\{A = true, B = false, D = true, E = true\}$	C1, C2, C3, C4, C5, C6 *true*			**return** *true*

Therefore, $(A \vee B) \wedge (\neg B \vee \neg C) \wedge (C \vee A) \wedge (\neg D \vee E) \wedge (D \vee \neg E) \wedge D$ is satisfiable with the (partial) model $\{A = true, B = false, D = true, E = true\}$.

The models that satisfy $(A \vee B) \wedge (\neg B \vee \neg C) \wedge (C \vee A) \wedge (\neg D \vee E) \wedge (D \vee \neg E) \wedge D$ are:

$\{A = true, B = false, C=true, D = true, E = true\}$,
$\{A = true, B = false, C=false, D = true, E = true\}$.

Example 2:
To check the satisfiability of $A \wedge \neg A$:

Clauses:

C1: A
C2: $\neg A$

Initial call: DPLL(*clauses*, [A], {})

Function Call	***symbols***	***model***	**Clause status**	**Pure symbol**	**Unit clause**	**Action (Assignment)**
1	A	{}	None *true*, None *false*	-	A (C1)	$A = true$
2	{}	$\{A = true\}$	C2 *false*			**return** *false*

Therefore, $A \wedge \neg A$ is not satisfiable.

Example 3:
To check the satisfiability of $(A \vee B) \wedge (\neg A \vee B) \wedge (\neg A \vee \neg B)$:

Clauses:

C1: $A \vee B$
C2: $\neg A \vee B$
C3:$\neg A \vee \neg B$

Initial call: DPLL(*clauses*, $[A, B]$, $\{\}$)

Function Call	***symbols***	***model***	**Clause status**	**Pure symbol**	**Unit clause**	**Action (Assignment)**
1	A, B	{}	None *true*, None *false*	-	-	Branch on A **Branch 1:** $A = true$ **Branch 2:** $A = false$
2 **(Branch 1)**	B	$\{A = true\}$	C1 *true*	-	B (C2)	$B = true$
3	[]	$\{A = true, B = true\}$	C1, C2 *true*, C3 *false*			**return** *false*
4 **(Branch 2)**	B	$\{A = false\}$	C2, C3 *true*	B (positive)		$B = true$
5	[]	$\{A = false, B = true\}$	C1, C2, C3 *true*			**return** *true*

Back at the root (initial call):

false (**Branch 1**) OR ***true*** (**Branch 2**) = ***true***

Therefore, $(A \vee B) \wedge (\neg A \vee B) \wedge (\neg A \vee \neg B)$ is satisfiable.

The only model that satisfies $(A \vee B) \wedge (\neg A \vee B) \wedge (\neg A \vee \neg B)$ is:
$\{A = false, B = true\}$

Note that DPLL algorithm can be used to check entailment.

In order to check if $KB \models \alpha$, the statement $KB \wedge \neg\alpha$ can be checked for satisfiability by the DPLL algorithm; because $KB \models \alpha$ if and only if $KB \wedge \neg\alpha$ is not satisfiable.

If the DPLL algorithm returns *false*, then $KB \models \alpha$.
If the DPLL algorithm returns *true*, then $KB \not\models \alpha$.

Definite Clauses and Horn Clauses

A clause in which **exactly one** literal is positive is a **definite clause**.

Examples: $\neg P \vee \neg Q \vee R$, $P \vee \neg Q$, etc.

- $P \vee \neg Q \vee R$ is NOT a definite clause.

A clause in which **at most one** literal is positive is a **Horn clause**.

Examples: $\neg P \vee \neg Q \vee R$, $\neg P \vee \neg Q$, etc.

- $P \vee \neg Q \vee R$ is NOT a Horn clause.

Note:

- Every definite clause is a Horn clause.

- Horn clauses are closed under resolution, that is, resolving two Horn clauses yields a Horn clause.

Example:

$(\neg P \vee \neg Q \vee R) \wedge (\neg P \vee \neg Q \vee \neg R) \vDash \neg P \vee \neg Q$

(Horn clause) (Horn clause) (Horn clause)

Every Horn clause can be written as an implication.

Examples:

1. Horn clause with multiple literals among which one is positive (also a definite clause):

$$\neg P \vee \neg Q \vee R \equiv \neg(P \wedge Q) \vee R \equiv (P \wedge Q) \Rightarrow R$$

2. Horn clause with a single literal that is positive (also a definite clause):

$$P \equiv \mathbf{T} \Rightarrow P$$

3. Horn clause with no positive literals:

$$\neg P \vee \neg Q \equiv \neg(P \wedge Q) \equiv (P \wedge Q) \Rightarrow \mathbf{F}$$

$$\neg P \equiv P \Rightarrow \mathbf{F}$$

Forward Chaining Algorithm

function PL-FC-ENTAILS?(*KB*, *q*) **returns** *true* or *false*
  inputs: *KB*, the knowledge base, a set of propositional definite clauses
    q, the query, a proposition symbol
  count ← a table, where *count*[c] is initially the number of symbols in clause *c*'s premise
  inferred ← a table, where *inferred*[*s*] is initially *false* for all symbols
  queue ← a queue of symbols, initially symbols known to be true in *KB*

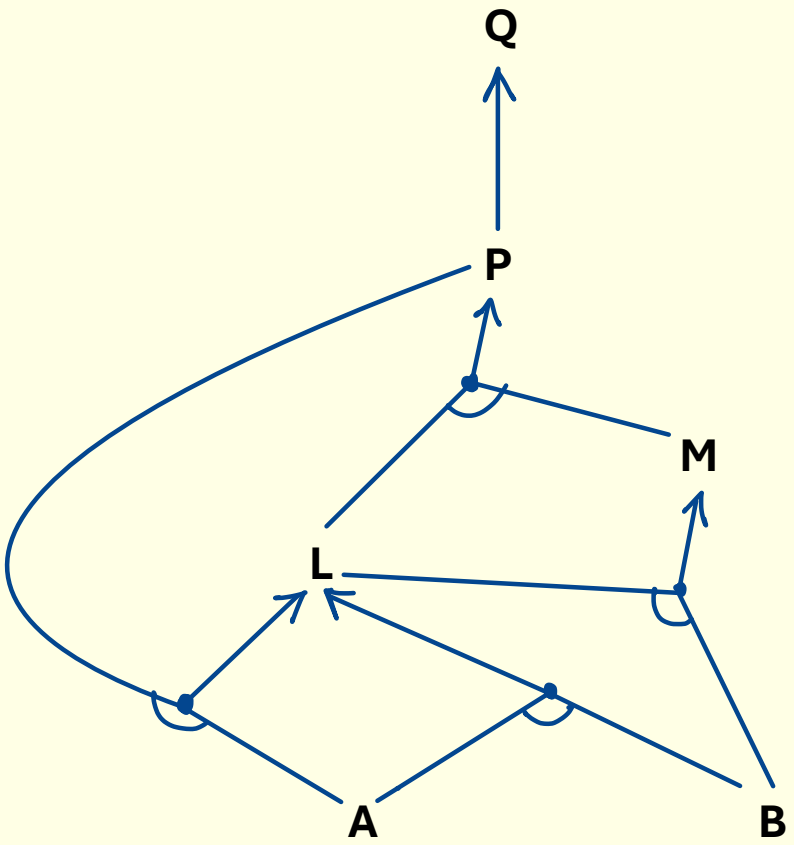
  while *queue* is not empty **do**
    p ← POP(*queue*)
    if *p* = *q* **then return** *true*
    if *inferred*[*p*] = *false* **then**
      inferred[*p*] ← *true*
      for each clause *c* in *KB* where *p* is in *c*.PREMISE **do**
        decrement *count*[c]
        if *count*[*c*] = 0 **then** add c.CONCLUSION to *queue*
  return *false*

Example:

KB: $A, B, \neg A \vee \neg B \vee L, \neg A \vee \neg P \vee L, \neg B \vee \neg L \vee M, \neg L \vee \neg M \vee P, \neg P \vee Q$.
Query q: Q

The implications corresponding to the clauses in KB are:
A (fact), B (fact), $A \wedge B \Rightarrow L$, $A \wedge P \Rightarrow L$, $B \wedge L \Rightarrow M$, $L \wedge M \Rightarrow P$, $P \Rightarrow Q$

AND-OR Graph corresponding to the definite clauses in the KB:



inferred [*s*] = *false* for all symbols *s*.
queue = {*A*, *B*}

1) queue $\neq \emptyset$.
   $p \leftarrow A$
   inferred [A] $\leftarrow$ true
2) queue $\neq \emptyset$.
   $p \leftarrow B$
   inferred [B] $\leftarrow$ true
   add L to queue.
3) queue $\neq \emptyset$.
   $p \leftarrow L$.
   inferred [L] $\leftarrow$ true
   add M to queue.
4) queue $\neq \emptyset$.
   $p \leftarrow M$.
   inferred [M] $\leftarrow$ true.
   add P to queue.
5) queue $\neq \emptyset$.
   $p \leftarrow P$.
   inferred [P] $\leftarrow$ true.
   add Q to queue.
6) queue $\neq \emptyset$.
   $p \leftarrow Q$.
   $p = q$; return true.

Clause (as implication)	count
$P \Rightarrow Q$	~~1~~ 0
$L \wedge M \Rightarrow P$	~~2~~ ~~1~~ 0
$B \wedge L \Rightarrow M$	~~2~~ ~~1~~ 0
$A \wedge P \Rightarrow L$	~~2~~ 1
$A \wedge B \Rightarrow L$	~~2~~ ~~1~~ 0
A	0
B	0

Backward Chaining

Idea: Work backwards from the query q.

To prove q by backward chaining:

If the query q is known to be true, then there is nothing to be done.
Else, prove by backward chaining all premises of some implication in KB whose conclusion is q. (Each premise to be proved is a subgoal.)

For every new subgoal:
(i) Avoid loops: Check if the new subgoal is already on goal stack.
(ii) Avoid repeated work: Check if the new subgoal has already been proved true, or has already failed.

If all subgoals are proven from facts/ other rules, then the goal is proven.
Otherwise, the attempt fails.

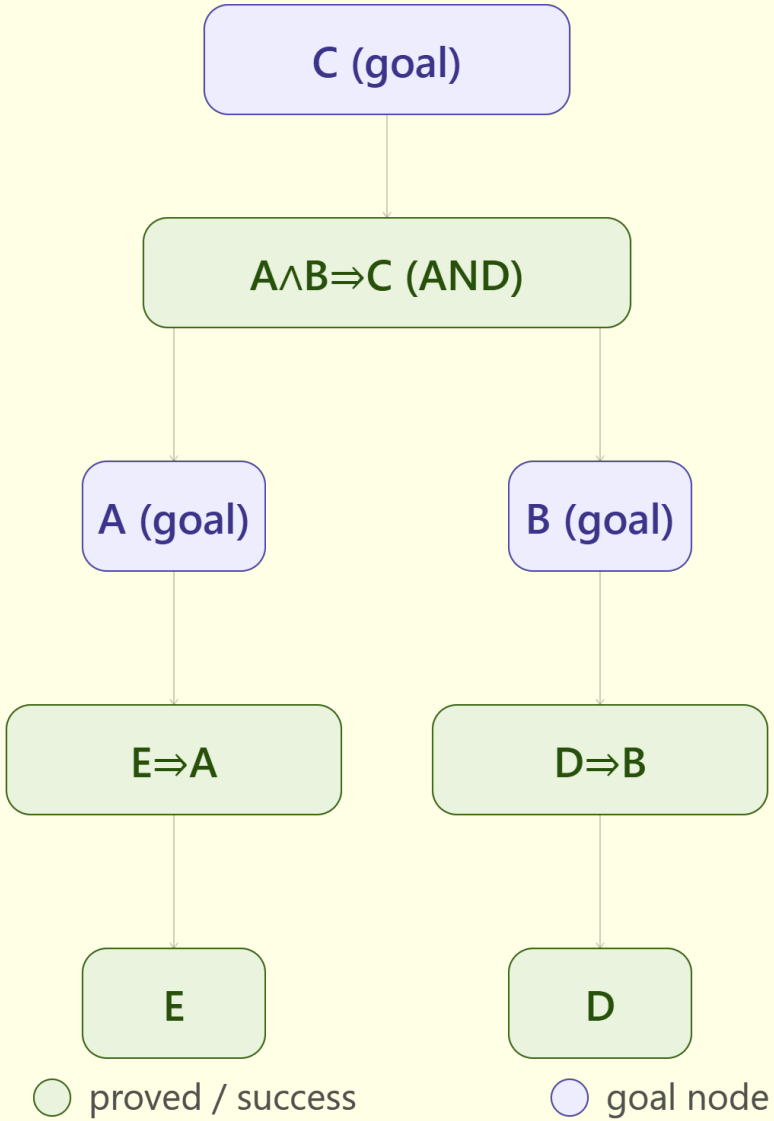
function PL-BC-ENTAILS?(*KB*, *q*) **returns** *true* or *false*
  inputs: *KB*, the knowledge base, a set of propositional definite clauses
  q, the query, a proposition symbol
  return BC-OR(*KB*, *q*, {}) ({} is the empty set of goals already on the path)

function BC-OR(*KB*, *q*, *path*) **returns** *true* or *false*
  inputs: *KB*, the knowledge base, a set of propositional definite clauses
  q, the current goal, a proposition symbol
  path, a set of symbols already being proved (for cycle detection)
  if *q* is known to be *true* in *KB* **then return** *true*
  if *q* ∈ *path* **then return** *false* (If q depends on itself, return false)
  for each clause *c* in *KB* where *q* = c.CONCLUSION **do**
    if BC-AND(*KB*, c.PREMISE, {*q*} ∪ *path*) **then return** *true*
  return *false*

function BC-AND(*KB*, *premises*, *path*) **returns** *true* or *false*
  inputs: *KB*, the knowledge base, a set of propositional definite clauses
  premises, a list of proposition symbols (one clause's premise)
  path, a set of symbols already being proved
  if *premises* = {} **then return** *true*
  p ← FIRST(*premises*)
  rest ← REST(*premises*)
  return BC-OR(*KB*, *p*, *path*) **and** BC-AND(*KB*, *rest*, *path*)

Example 1:

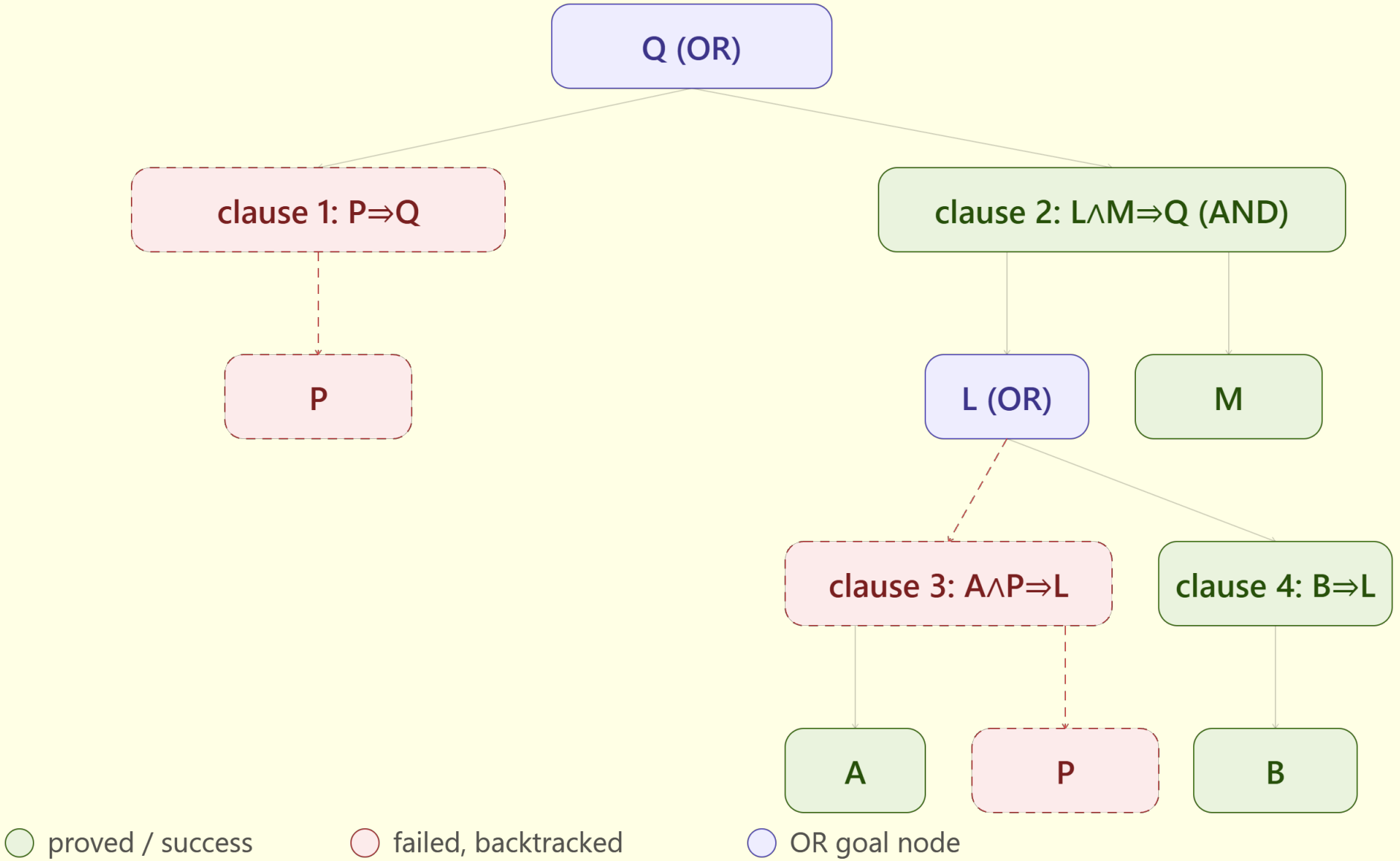
KB: $D, E, \ A \wedge B \Rightarrow C, \ D \Rightarrow B, \ E \Rightarrow A$
Query q: C


C (goal)
A∧B⇒C (AND)
A (goal)
B (goal)
E⇒A
D⇒B
E
D
proved / success
goal node

Example 2:

KB: $A, B, M, \neg P \vee Q, \neg L \vee \neg M \vee Q, \neg A \vee \neg P \vee L, \neg B \vee L$
Query q: Q

The implications corresponding to the clauses in KB are the following:
$P \Rightarrow Q$ (clause 1)
$L \wedge M \Rightarrow Q$ (clause 2)
$A \wedge P \Rightarrow L$ (clause 3)
$B \Rightarrow L$ (clause 4)
$\boldsymbol{T} \Rightarrow A$ (clause 5)
$\boldsymbol{T} \Rightarrow B$ (clause 6)
$\boldsymbol{T} \Rightarrow M$ (clause 7)


Q (OR)
clause 1: P⇒Q
clause 2: L∧M⇒Q (AND)
P
L (OR)
M
clause 3: A∧P⇒L
clause 4: B⇒L
A
P
B
proved / success
failed, backtracked
OR goal node

Backward chaining does a Depth-First Search of the AND-OR graph of the KB, starting from the node corresponding to the query.

Comparison between Forward Chaining and Backward Chaining

- Forward chaining is data-driven.
- Backward chaining is goal-driven.

- The time complexity of forward chaining is linear in the size of the KB (total number of literals in all Definite/Horn clauses).
- The time complexity of backward chaining *can be* much less than linear in the size of the KB (In the worst case, it can be linear in the size of the KB.)